

QUANT TRADING SYSTEM

Architecture, Quantitative Finance & Backend Engineering Guide

Role: Quantitative Developer (Execution Engine & Backtesting)

Target Firm: Vera Developers Placement Process

Focus: Clear, from-the-ground-up explanation of algorithmic execution, causal technical indicators, Indian market microstructure, deterministic order idempotency, and lookahead-free backtesting.

Note for Full-Stack & Frontend Developers Learning Quant Backend:

If you come from a frontend or traditional full-stack background, quant trading backends differ in one crucial way: *mathematical determinism and zero tolerance for temporal leakage*. In web apps, a 10ms network delay or reading from state slightly early is harmless. In quantitative execution, looking 1 bar into the future ('lookahead bias') or counting pending orders as inventory creates phantom profits on paper that blow up with real capital. This guide explains every concept simply, logically, and with real-world analogies.

Chapter	Topic Covered	Core Takeaway
1. Executive Overview	System Architecture & Data Flow	How frontend & backend connect without drift
2. Execution Engine	Grid, Pyramiding, Stops, Kill Switch	Dynamic ATR spacing and risk capping
3. Causal Indicators	EMA, RSI, ATR, and OBV	Why causal math prevents lookahead bias
4. Indian Market Plumbing	Slippage, STT/CTT, Brokerage, GST	Matching returns down to the paisa
5. Order Management	Idempotency, SHA-1, Crash Reconciliation	Restart-safe state machine & position truth
6. Backtest Harness	Bar-Accurate Fills & Walk-Forward	Signal at bar i -> Fill at bar i+1 open
7. Interview Prep	Talking Points for the Technical Round	How to explain your design decisions cleanly

1. Executive Overview & System Architecture

At its core, this project implements a **systematic, algorithmic quantitative trading system**. A quantitative trading system doesn't rely on human gut feeling; it takes historical or live price bars (Open, High, Low, Close, Volume), calculates mathematical indicators, makes automated buy/sell decisions, manages open risk, and accounts for real-world exchange costs down to the paisa.

The Big Picture: How Data Flows Through the System

Imagine an assembly line. Every time a new 5-minute candle closes in the market, the following 6-step sequence occurs:

Step	Component	What Happens in Plain English
1. Ingestion	src/data.py	Receives market OHLCV bars (either synthetic generator or Zerodha Kite Connect feed).
2. Indicators	src/indicators.py	Computes Trend (EMA), Momentum (RSI), Volatility (ATR), and Volume (OBV) strictly using past data.
3. Strategy	src/engine.py	GridReverseEngine.on_bar() evaluates whether to enter, add a pyramid unit, or stop-and-reverse.
4. Order Mgmt	src/orders.py	Emits orders with deterministic SHA-1 IDs to ensure duplicate submissions are impossible.
5. Fills & Costs	src/costs.py + backtest.py	Fills the order at the NEXT bar's OPEN, subtracting slippage, flat $\blacksquare$ 20 brokerage, STT, and GST.
6. Presentation	frontend/ (React)	Visualizes equity curves, trade blotters, and indicators on an institutional Bloomberg-style UI.

Why We Built Both Python Backend & React Frontend

In algorithmic trading desks, research is done in Python (pandas, numpy, vectorbt), but portfolio managers and risk officers monitor performance through high-density graphical dashboards. By implementing both a robust Python core and a standalone React dashboard, the project achieves two critical goals:

- Recruiter Convenience:** A recruiter can open a Vercel link and interact with the strategy in 5 seconds.
- Architecture Rigor:** The React frontend mirrors the Python logic byte-for-byte, verifying that the engineering principles hold true across language boundaries.

2. The Execution Engine (src/engine.py)

The execution engine is the 'brain' of the strategy. It answers one question: *Given the current price and market volatility, what action should we take right now?*

Concept 1: Trend Filter (Fast EMA vs Slow EMA)

Before making any trade, the engine checks market direction. It compares a fast 12-period Exponential Moving Average against a slow 26-period Exponential Moving Average. If **EMA(12) > EMA(26)**, the regime is BULLISH; if **EMA(12) < EMA(26)**, the regime is BEARISH. When flat (no open position), the engine initiates a trade matching this trend direction.

Concept 2: ATR-Based Grid Spacing & Pyramiding

What is Pyramiding? Pyramiding means adding more units to an already profitable trade. Rather than betting everything at once, we scale into a winning position as the market moves in our favor.

Why use ATR (Average True Range) spacing instead of fixed points? If you set a fixed spacing rule of $\$10$, it might work when the stock moves $\$15$ a day, but fail completely when volatility expands and it moves $\$100$ a day. By setting $\text{grid_spacing} = \text{atr_multiplier} * \text{ATR}$, the grid automatically expands during high volatility and tightens during quiet consolidation.

Concept 3: Position Hard Cap

Uncontrolled pyramiding can cause catastrophic losses if a trend suddenly reverses when you hold a huge position. The engine enforces a strict **Position Cap** (e.g. maximum 5 units). Once the cap is reached, further pyramid signals are silently rejected by risk control.

Concept 4: Stop-and-Reverse (SAR)

A regular stop-loss flattens your position to cash. A **Stop-and-Reverse** does something smarter: it recognizes that if the market moves against you by a large distance ($\text{stop_multiplier} * \text{ATR}$), your original directional thesis was completely wrong and a strong counter-trend has likely begun. The engine issues two orders at once: first, it sells all existing long units to flatten, and immediately opens a new short unit to ride the new trend downwards.

Concept 5: Portfolio Kill Switch

Algorithmic trading desks mandate an automated circuit breaker. The engine continuously tracks **High-Water Mark Equity** (the highest peak portfolio value ever reached). If mark-to-market drawdown from that peak exceeds 15% ($\text{kill_switch_dd_pct} = 0.15$), the Kill Switch fires: it immediately cancels all pending orders, flattens all open inventory to cash, and locks out the engine from opening any new positions.

3. Technical Analysis Indicators (src/indicators.py)

The brief mandated: 'one tested implementation per indicator family, no duplicated math.' Furthermore, every indicator must be strictly **causal**.

What Does 'Causal' & 'No Lookahead' Mean?

In traditional programming, functions can inspect any element in an array. But in time-series finance, bar *i* represents the market at 10:00 AM, and bar *i*+1 is 10:05 AM. If the indicator at 10:00 AM uses data from 10:05 AM, it is guilty of **Lookahead Bias**. A backtest with lookahead bias will show unbelievable 1,000% returns because it effectively knows the future, but will immediately lose all capital in live trading. In our codebase, every indicator value at index *i* is computed strictly from data where index $\leq i$.

Family	Indicator & Formula	Role in Execution Engine
Trend	EMA (Exponential Moving Average) $val[i] = \alpha * price[i] + (1-\alpha) * val[i-1]$ where $\alpha = 2 / (period + 1)$	Fast EMA(12) vs Slow EMA(26) cross determines initial entry direction (Long vs Short).
Momentum	RSI (Relative Strength Index) Wilder's Smoothing: $\alpha = 1 / period$ $RS = avg_gain / avg_loss$ $RSI = 100 - (100 / (1 + RS))$	Measures velocity of directional price movement. Bounded between 0 and 100.
Volatility	ATR (Average True Range) $TR = \max(H-L, H-Cp , L-Cp)$ Wilder's smoothing of True Range over 14 bars.	The single source of truth for volatility. Sets dynamic grid add spacing and stop-loss distance.
Volume	OBV (On-Balance Volume) $OBV[i] = OBV[i-1] + sign(\Delta Close) * Vol[i]$	Tracks institutional accumulation and distribution by adding volume on up-bars and subtracting on down-bars.

4. Indian Market Plumbing & Statutory Costs (src/costs.py)

A hallmark of an amateur trading system is ignoring transaction costs. On paper, high-frequency grid additions look profitable, but in the real world, exchange fees, taxes, and slippage will bleed the account dry. Our cost engine models Indian financial plumbing down to the paisa.

Charge Component	Standard Rate in India	How It Is Computed in src/costs.py
Slippage	2.0 bps (0.02%)	Market impact: BUY fills higher than quoted price; SELL fills lower. $adj = price * (2 / 10,000)$.
Flat Brokerage	■20.00 per executed order	Zerodha Kite Connect discount-broker model: fixed flat fee per trade execution.
STT / CTT	1.0 bps (Sell side only)	Securities/Commodities Transaction Tax: Statutory tax levied strictly on SELL turnover in derivatives.
Exchange Charges	0.35 bps (Turnover)	NSE / MCX exchange infrastructure turnover charges applied on both BUY and SELL sides.
GST (Govt Tax)	18.0% on Service Fees	$GST = (Brokerage + Exchange Charges) * 0.18$. Note: GST applies to service fees, NOT turnover.

5. Order & State Management (src/orders.py)

In production trading, software crashes, network sockets drop, and servers reboot. What happens if your trading engine places a BUY order, and the server loses power 1 millisecond later? When the server boots back up, will it submit that order a second time and buy twice as much? In src/orders.py, we solve this with **Deterministic Idempotency** and **Broker Reconciliation**.

Concept 1: Deterministic Client Order IDs (SHA-1)

Instead of using random UUIDs (e.g. uuid4()), every order generated by the engine has its client_order_id computed as a deterministic cryptographic hash:

```
client_order_id = sha1(f'{strategy_id}|{bar_index}|{intent}')[16]
```

Why is this revolutionary? If bar 46 emits a stop-and-reverse order, its ID will always be 3a98e1f04bc8d821. If the engine crashes, reboots, and replays bar 46, it sees that 3a98e1f04bc8d821 already exists in the book! It simply returns the existing order instead of double-submitting. Replay is mathematically idempotent.

Concept 2: Position Truth from Fills Only

A common amateur bug is updating position inventory when an order is *placed* (intent). If that order is rejected or cancelled by the broker, the system's internal position drifts from reality. In our system, OrderManager.net_position() iterates over the order book and sums **strictly orders with status == FILLED**. Intentions never inflate risk truth.

Concept 3: Broker Reconciliation Algorithm

Upon restarting after a crash, the engine calls OrderManager.reconcile(broker_snapshot):

- Adopts Unknown Orders:** If an order was filled on the broker side while local disk was crashing, the engine adopts it as truth.
- Resolves Status Mismatches:** If local state recorded an order as PENDING, but broker snapshot shows FILLED, the local state is updated to FILLED with the true broker execution price.
- Generates Audit Notes:** Every discrepancy produces a structured log for compliance monitoring.

6. Backtest Harness & Walk-Forward Validation (src/backtest.py)

A backtest harness is a simulation engine that tests how a strategy would have performed on historical data. Our backtester enforces three non-negotiable quantitative standards:

Rule 1: Fills at Bar (i+1) Open (Zero Lookahead)

If an indicator cross occurs on bar i at 11:30 AM close, you **cannot** fill at bar i's close. In the real world, bar i's close is only known once the bar has finished! Therefore, any order triggered at bar i close executes at bar i+1's OPEN (11:30:01 AM). This 1-bar execution delay reflects physical reality.

Rule 2: Indicator Warmup Discipline

A 26-period EMA requires at least 26 bars of data before its mathematical value is valid. During bars 0 to 25, indicator values are genuine NaN. The engine refuses to trade until all indicators are fully warmed up, avoiding garbage signals on startup.

Rule 3: Walk-Forward Validation with Hard Fold Isolation

In machine learning and quant research, strategies often suffer from 'overfitting' (memorizing past noise). We divide our 600 bars into **4 chronological, non-overlapping test folds**. For each fold, we instantiate a brand new engine and order manager from scratch. No position inventory or indicator state is allowed to leak across fold boundaries.

Metric	Formula	Financial Meaning
Total Return %	$((\text{Final_Equity} / \text{Initial_Cash}) - 1) * 100$	Net percentage gain or loss after all trading fees and slippage.

Max Drawdown %	$\min((\text{Equity}[i] - \text{Peak_Equity}[i]) / \text{Peak_Equity}[i]) * 100$	The worst peak-to-trough drop in account value. Measures downside pain and risk of ruin.
Sharpe Ratio	$(\text{Mean}(\text{Returns}) / \text{Std}(\text{Returns})) * \text{sqrt}(252 * 75)$	Risk-adjusted return. Compares excess return to volatility. Annualized for 5-min bars (252 days × 75 bars/day).

7. Interview Prep: What to Tell the Recruiter

When interviewing for a Quant Developer role, interviewers care about your **engineering judgment**, not just syntax. Here are the 5 questions they will ask and how this project demonstrates mastery:

Q1: 'How do you ensure backtest numbers reconcile to live trading numbers?'

Answer: 'By having one single decision function with two callers. In my implementation, `GridReverseEngine.on_bar()` is the single source of truth for strategy logic. The backtester drives it in a loop; a live `WebSocket` bar-close handler drives the exact same method. Fills in the backtester are strictly deferred to bar $i+1$ open with identical `CostModel` slippage, flat $\blacksquare 20$ brokerage, STT, and GST, preventing any backtest-to-live drift.'

Q2: 'How does your engine handle unexpected process crashes or restarts?'

Answer: 'Orders use deterministic idempotency keys: `client_order_id = sha1(f'{{strategy}}{bar}{{intent}}')`. If the engine crashes and restarts, re-submitting the same decision produces the exact same hash, which is a safe no-op in `OrderManager`. Furthermore, `OrderManager.reconcile()` takes a broker snapshot, adopts any orders filled during the crash window, corrects status mismatches, and calculates net position truth strictly from FILLED orders.'

Q3: 'Why did you use ATR for grid spacing and stops instead of fixed rupee targets?'

Answer: 'Market volatility is non-stationary. A fixed $\blacksquare 10$ grid might trade well in low-volatility consolidation but get chopped up when volatility expands to $\blacksquare 50$. Dynamic ATR spacing expands grid distance and stops during high volatility regimes and tightens during quiet regimes, preserving risk-reward consistency across time.'

Q4: 'Why is there both a Python engine and a React frontend?'

Answer: 'Algorithmic development happens in Python for numerical discipline and `pytest` coverage, but visual monitoring is critical for risk management. I built an institutional React dashboard with an embedded client-side simulation engine so the recruiter can test parameters, walk-forward folds, and crash recovery live in their browser without needing cloud compute.'

Q5: 'What testing discipline did you apply?'

Answer: '39 automated tests in `pytest` with zero external mock leaks. Tests cover indicator hand-calculations, one-sided STT tax rules, position cap enforcement, kill switch firing on extreme drawdown, and a full simulated crash reconciliation regression test asserting net position parity after restart.'

Summary Checklist for Your Submission:

- **39 Pytest tests pass:** Run `python -m pytest tests/ -v`
- **Frontend is live:** Open `http://localhost:3001` to view and demo the dashboard
- **Deployable to Vercel:** Follow instructions in `DEPLOYMENT.md` for a public recruiter link
- **PDF Reports ready:** Both the Technical Submission Report and this Learning Guide are in report/.